

高级语言程序设计

实验报告

南开大学 工科试验班

姓名 章开元

学号 2511321

班级 3-4

2026 年 5 月 12 日

Contents

1 作业题目	3
2 开发软件	3
3 课题要求	3
4 主要流程	3
4.1 整体流程	3
5 主要流程	3
5.1 整体流程	3
5.2 可视化模式主循环流程	4
5.3 可视化模式主循环流程	4
6 项目架构	6
6.1 目录结构总览	6
7 核心组件	7
7.1 主要的类	7
7.1.1 逻辑层核心类	7
7.1.2 渲染层核心类	8
7.1.3 应用层与网络层核心类	8
7.2 类的继承关系	8
8 核心算法	9
8.1 A* 网格寻路算法	9
8.1.1 算法流程	9
8.1.2 性能优化	10
8.2 僵尸 AI 有限状态机	10
8.2.1 状态转换图	10
8.2.2 三种状态的行为	10
8.3 定点数运算	10
8.4 战斗系统	11
8.4.1 近战流程	11
8.4.2 远程（炮兵）流程	11
8.5 战争迷雾	11
9 单元测试	11
9.1 测试内容	11
9.1.1 基础层测试（5 个）	11
9.1.2 核心系统测试（8 个）	12
9.1.3 集成测试（3 个）与专项测试（5 个）	12
9.2 测试结果	13
10 收获	13
10.1 C++ 工程能力	13
10.2 软件架构设计	14
10.3 游戏开发基础	14
10.4 算法与数据结构	14
10.5 测试与质量保证	14

10.6 调试与问题排查	14
11 AI 使用报告	14

1 作业题目

融合植物大战僵尸元素和红警框架的即时战略游戏——**The Chlorophyll Protocol (叶绿素协议)**

2 开发软件

- **开发环境：** macOS, Visual Studio Code
- **构建系统：** CMake 3.21+
- **编译器：** Clang (Apple Clang), C++20 标准
- **图形库：** SFML 2.6 (Simple and Fast Multimedia Library)
- **版本控制：** Git
- **测试框架：** CTest (GoogleTest 可选)
- **代码检查：** Python 3 (用于检查逻辑层无浮点类型)

3 课题要求

- **确定性模拟：** 采用固定时间步长 (30 tick/s) 的模拟循环，保证相同输入产生完全相同的游戏状态，为多人在线对战奠定基础。
- **ECS 架构：** 手写 Entity-Component-System 框架，实体为纯 ID，组件为纯数据结构，系统按固定 Phase 顺序执行。
- **分层设计：** 严格分离逻辑层 (`src/logic/`)、渲染层 (`src/render/`)、应用层 (`src/app/`) 和网络层 (`src/net/`)，逻辑层不得使用浮点类型。
- **即时战略核心玩法：** 实现建筑建造、单位生产、资源管理、近战/远程战斗、A* 寻路、战争迷雾等功能。
- **僵尸 AI 敌人：** 实现有限状态机驱动的自主敌人，周期性生成波次并攻击玩家基地。
- **可视化与交互：** 使用 SFML 实现等角投影地图渲染、RA2 风格 UI (侧边栏、资源栏、单位信息面板)。
- **单元测试覆盖：** 对核心系统编写测试，保证确定性、数值正确性和系统行为正确性。

4 主要流程

4.1 整体流程

程序启动后进入 `src/app/Main.cpp` 中的 `main()` 函数，支持三种运行模式：

5 主要流程

5.1 整体流程

程序启动后进入 `src/app/Main.cpp` 中的 `main()` 函数。系统首先进行命令行解析与核心控制器的生命周期构建，随后根据不同的启动参数分发至四个并行的底层运行驱动实例，最后输出全局状态哈希。具体执行拓扑树见图 1：

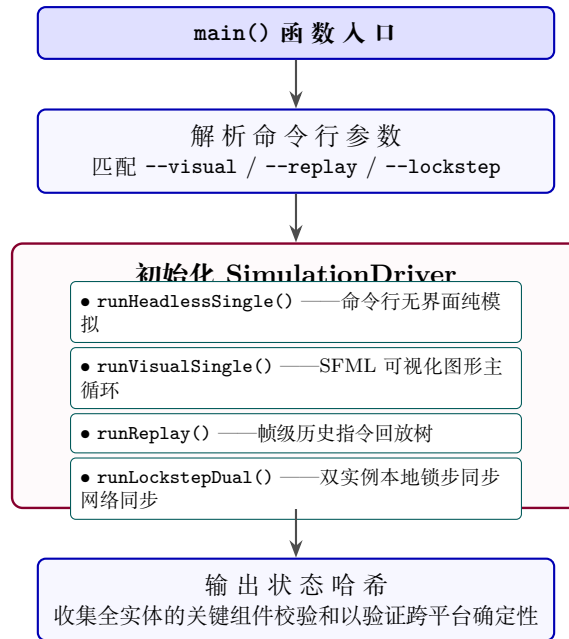


Figure 1: `main()` 初始化链路与模拟驱动分发拓扑图

5.2 可视化模式主循环流程

可视化模式 (`runVisualSingle()`) 的核心是一个基于真实墙钟时间 (Wall-Clock Time) 采样、并由 `GameLoop` 进行固定步长截断调度的双层嵌套循环。其执行细节如图 2 所示：

5.3 可视化模式主循环流程

可视化模式 (`runVisualSingle()`) 的核心是一个基于真实墙钟时间 (Wall-Clock Time) 采样、并由 `GameLoop` 进行固定步长截断调度的双层嵌套循环。其执行细节如图 2 所示：



Figure 2: 可视化驱动模式 `runVisualSingle()` 的渲染与逻辑 Tick 双层解耦主循环流程图

6 项目架构

项目采用严格的四层架构，各层之间职责明确，解耦良好。以下为系统架构的具体拓扑拓扑拓扑关系图：

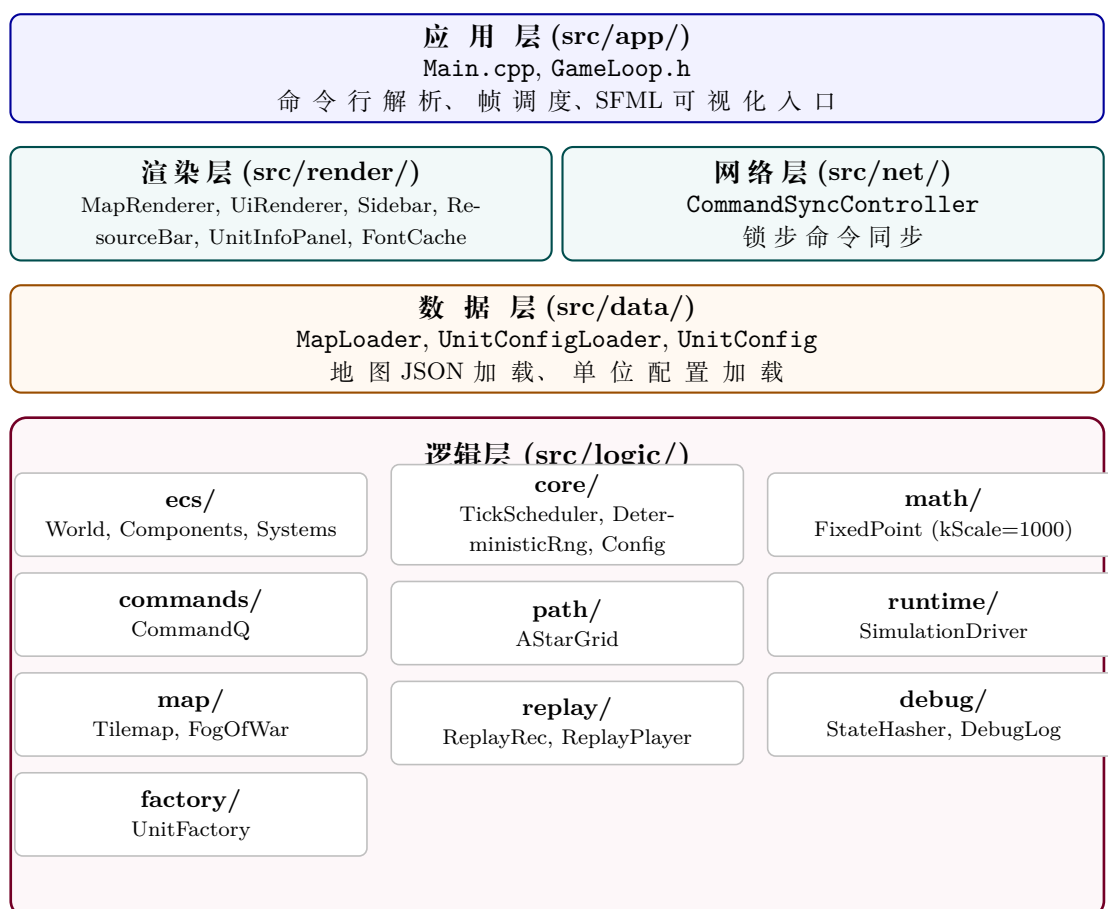


Figure 3: The Chlorophyll Protocol 项目系统架构四层拓扑图

6.1 目录结构总览

Table 1: 项目目录结构与文件分布总览

目录	文件数	说明
src/logic/ecs/	8	ECS 核心: World、Components、系统管线、僵尸 AI、波次生成
src/logic/core/	6	时间调度、确定性 RNG、模拟配置、稳定排序
src/logic/math/	2	定点数运算类
src/logic/path/	2	A* 网格寻路
src/logic/map/	4	地图数据结构、战争迷雾
src/logic/commands/	3	指令队列、玩家指令类型
src/logic/runtime/	2	模拟驱动器 (多模式分发)
src/logic/replay/	4	回放录制与播放
src/logic/debug/	3	状态哈希、调试日志
src/logic/factory/	2	单位工厂
src/data/	4	地图加载、单位配置加载
src/render/	12	SFML 渲染: 地图、UI、字体
src/net/	2	锁步网络同步
src/app/	2	入口和帧调度
src/tests/	17	单元测试和集成测试

7 核心组件

7.1 主要的类

项目中总共有约 15 个核心类，分布在各个层次中：

7.1.1 逻辑层核心类

类名	文件	职责
World	src/logic/ecs/World.h	ECS 世界容器: 实体管理、组件存储、系统注册与 tick 执行
FixedPoint	src/logic/math/FixedPoint.h	定点数运算 (kScale=1000), 替代浮点数保证确定性
TickScheduler	src/logic/core/TickScheduler.h	固定时间步长调度器, 支持追赶限制 (最多 5 tick)
DeterministicRng	src/logic/core/DeterministicRng.h	基于种子的确定性随机数生成器
CommandQueue	src/logic/commands/CommandQueue.h	每个实体的指令队列, 支持按 tick 排序执行
SimulationDriver	src/logic/runtime/SimulationDriver.h	模拟驱动器, 分派单机/回放/锁步三种模式
ReplayRecorder	src/logic/replay/ReplayRecorder.h	将玩家指令录制为回放文件
ReplayPlayer	src/logic/replay/ReplayPlayer.h	从回放文件读取并回放指令
StateHasher	src/logic/debug/StateHasher.h	对 World 状态计算确定性哈希, 用于验证同步

7.1.2 渲染层核心类

类名	文件	职责
MapRenderer	src/render/MapRenderer.h	等角投影地图渲染（菱形地块拼接）
UiRenderer	src/render/UiRenderer.h	UI 管理器：协调资源栏、侧边栏、单位信息面板，事件路由
Sidebar	src/render/Sidebar.h	侧边栏：建筑/防御/步兵/载具四个标签页的生产槽位
ResourceBar	src/render/ResourceBar.h	顶部资源栏：显示阳光和电力
UnitInfoPanel	src/render/UnitInfoPanel.h	选中单位信息面板：生命值、建造进度、武器模式
FontCache	src/render/FontCache.h	字体缓存：加载和管理 SFML 字体资源

7.1.3 应用层与网络层核心类

类名	文件	职责
GameLoop	src/app/GameLoop.h	帧循环桥接：将 wall-clock 时间转换为固定 tick 步进
CommandSyncController	src/net/CommandSyncController.h	锁步指令同步控制器

7.2 类的继承关系

本项目的设计以组合优于继承为原则，类之间主要通过组合和依赖注入协作，而非继承。具体关系如下：

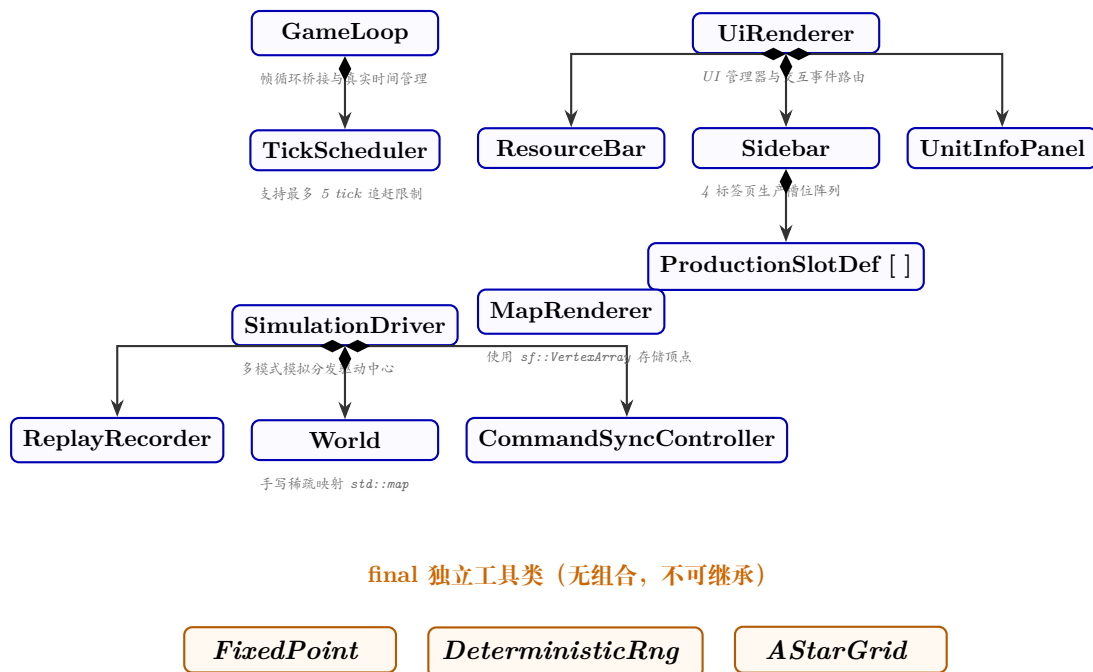


Figure 4: The Chlorophyll Protocol 核心类强组合关系（UML Composition）图

关键设计决策：所有逻辑层类均标记为 `final`，不允许继承。这样做的好处是：

1. 编译器可以进行去虚拟化优化（Devirtualization）。
2. 避免不必要的抽象层次。

3. 保持代码简单直观，便于测试和维护。

关键设计决策：所有逻辑层类均标记为 `final`，不允许继承。这样做的好处是：

1. 编译器可以进行去虚拟化优化 (Devirtualization)。
2. 避免不必要的抽象层次。
3. 保持代码简单直观，便于测试和维护。

8 核心算法

8.1 A* 网格寻路算法

位置： `src/logic/path/AStarGrid.cpp` (第 42–142 行)

8.1.1 算法流程

输入：起点 `GridCoord`、终点 `GridCoord`、障碍物集合、边界

输出：最短路径 `std::vector<GridCoord>`

1. **边界检查：**起点或终点越界则返回空路径。
2. **障碍物检查：**终点被阻挡则返回空路径。
3. **起点 = 终点：**直接返回单点路径。
4. 构建 `blockedGrid` 扁平数组 ($\mathcal{O}(1)$ 查找替代 $\mathcal{O}(\log n)$ `set` 查找)。
5. **初始化：**
 - $gScore[start] = 0$
 - $fScore[start] = manhattan(start, goal)$
 - $openHeap = [(fScore, start)]$ ，使用 `std::push_heap` 维护最小堆。
6. **主循环：**
`while openHeap 非空：`
 - (a) `pop` 堆顶 (当前 $fScore$ 最小的节点)。
 - (b) 若 $node == goal$ ，回溯重建路径并返回。
 - (c) 将 $node$ 加入 `closedSet`。
 - (d) 遍历 4 个邻居 (上下左右，四连通)：
 - 若邻居被阻挡或已在 `closedSet`，跳过。
 - 计算 $tentativeG = currentG + 1$ 。
 - 若 $tentativeG < bestKnown[neighbor]$ ：
更新 `cameFrom`、 $gScore$ 、 $fScore$ ，将邻居推入 `openHeap`。
7. 若 `openHeap` 耗尽仍未找到，返回空路径。

启发式函数：曼哈顿距离 $|dx| + |dy|$ (四连通网格下是 `admissible` 的，保证最短路径)。

8.1.2 性能优化

- 障碍物集合从 `std::set` 转换为扁平 `std::vector<bool>`，查找从 $\mathcal{O}(\log n)$ 降为 $\mathcal{O}(1)$ 。
- 使用 `std::push_heap/std::pop_heap` 维护优先队列，避免 `std::priority_queue` 无法更新已存在元素的限制。
- 重复推入的过时条目通过 `closedSet` 检测跳过。

8.2 僵尸 AI 有限状态机

位置： `src/logic/ecs/systems/ZombieAISystem.cpp`（第 27–181 行）

8.2.1 状态转换图

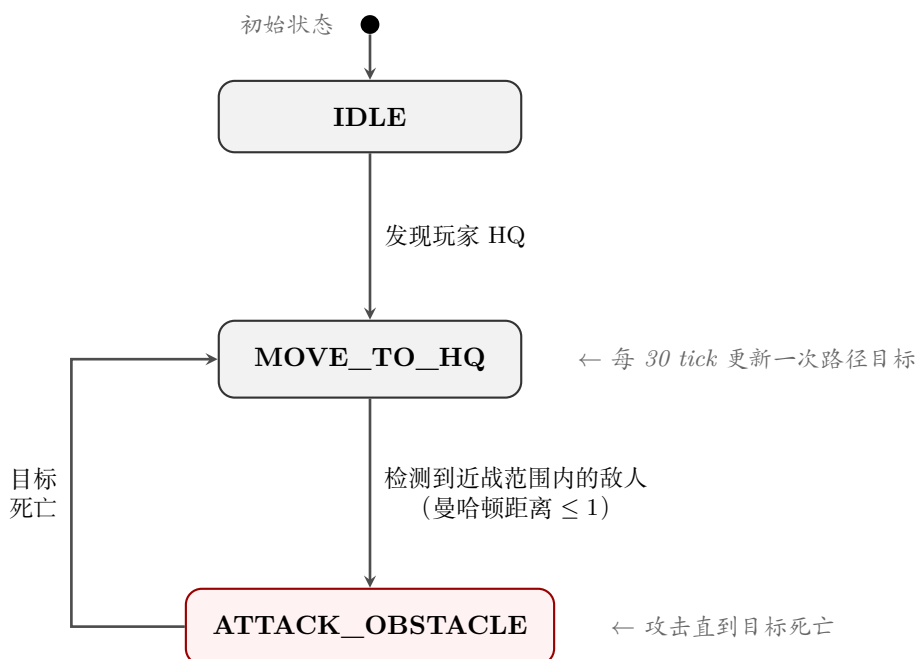


Figure 5: 僵尸 AI 有限状态 FSM 逻辑转换拓扑图

8.2.2 三种状态的行为

状态	行为	触发转换的条件
IDLE	不做任何操作	发现存活的玩家 HQ
MOVE_TO_HQ	每 30 tick 更新移动目标为 HQ 坐标，扫描近战范围内的敌人	检测到敌人 → ATTACK_OBSTACLE; HQ 丢失 → IDLE
ATTACK_OBSTACLE	停止移动（速度清零），持续攻击目标	目标死亡或消失 → MOVE_TO_HQ

8.3 定点数运算

位置： `src/logic/math/FixedPoint.h`

设计：使用 `int32_t` 存储，缩放因子 `kScale = 1000`。例如：

- `fromInt(5)` → 内部值 5000（表示 5.000）

- `fromRaw(250)` → 内部值 250 (表示 0.250)
- `toIntTrunc()` → 直接截断取整

为什么用定点数：浮点数在不同平台/编译器下可能产生微小差异（如舍入方向），破坏锁步同步所需的确定性。定点数所有运算均为整数运算，结果完全可重现。

支持的运算： +、-、*、/、==、!=、<、<=、>、>=，运算符重载以保持代码可读性。

8.4 战斗系统

位置： `src/logic/ecs/systems/BuiltInSystems.cpp` (`runCombatPhase()`，第 742–947 行)

8.4.1 近战流程

1. 检查攻击者电力是否充足 (`PowerConsumer` 组件)。
2. 冷却递减。
3. 检查攻击目标是否有效 (存活、不同队伍、在武器射程内)。
4. 伤害计算: `mitigatedDamage = max(0, baseDamage - armor)`。
5. 扣血并附加受击闪烁效果。

8.4.2 远程 (炮兵) 流程

1. 玉米加农炮每 10 tick 扫描一次射程内最近的敌人。
2. 切换模式 (`kToggleButterMode`): 高爆模式 (1,000,000 伤害, AOE 半径 2000 raw) vs 黄油模式 (300,000 伤害, AOE 半径 2 格, 冰冻 150 tick)。
3. 发射弹道投射物 (`BallisticProjectile`)，经过 `projectileTravelTicks` (24 tick) 后引爆。
4. 引爆时对 AOE 范围内所有敌方实体施加伤害和冰冻效果，生成爆炸视觉特效实体。

8.5 战争迷雾

位置： `src/logic/map/FogOfWar.h` / `FogOfWar.cpp`

每个队伍维护独立的迷雾状态。具有 `Vision` 组件的实体每 tick 以自身位置为中心揭示周围格子。迷雾随时间逐渐重新覆盖 (`advanceTick()`)，仅在被 `Vision` 实体持续揭示时保持可见。

9 单元测试

9.1 测试内容

项目共有 21 个测试用例，覆盖从基础数据结构到整体游戏循环的各个层次：

9.1.1 基础层测试 (5 个)

测试名称	文件	测试内容
DeterminismSmoke	src/tests/ DeterminismSmokeTest.cpp	两次相同输入产生相同状态哈希, 验证模拟确定性
FixedPointMath	src/tests/FixedPointTest.cpp	定点数加减乘除、比较运算、边界值 (INT_MAX/INT_MIN)
DeterministicRngSequence	src/tests/ DeterministicRngTest.cpp	相同种子产生相同序列, 不同种子产生不同序列
TickSchedulerDeterminism	src/tests/TickSchedulerTest.cpp	帧调度器的 tick 追赶和限制逻辑
EcsWorldPipeline	src/tests/EcsWorldTest.cpp	实体创建/销毁、组件设置/读取、实体存在性检查

9.1.2 核心系统测试 (8 个)

测试名称	文件	测试内容
EcsCoreSystems	src/tests/ EcsCoreSystemsTest.cpp	建筑建造、单位生产、攻击指令处理
GameplayLoopCore	src/tests/GameplayLoopTest.cpp	完整 tick 循环: 建造 → 生产 → 战斗 → 胜负判定
PlacementAndPathfinding	src/tests/ PlacementAndPathfindingTest.cpp	建筑放置合法性检查、A* 最短路径
CornCannonBastion	src/tests/systems/ CornCannonBastionTest.cpp	玉米加农炮的建造、高爆/黄油模式切换、AOE 伤害、冰冻效果
CommandQueueOrder	src/tests/systems/ CommandQueueOrderTest.cpp	指令按 tick 排序执行、指令出队顺序
RuntimeTelemetry	src/tests/systems/ RuntimeTelemetryTest.cpp	TickTelemetry 统计数据正确性
LogicTickBudget	src/tests/systems/ LogicTickBudgetTest.cpp	单 tick 执行时间的性能基线
UnitConfigFactory	src/tests/ UnitConfigFactoryTest.cpp	单位配置 JSON 加载、工厂创建实体

9.1.3 集成测试 (3 个) 与专项测试 (5 个)

测试名称	文件	测试内容
DualInstanceDeterminism	src/tests/integration/ DualInstanceDeterminismTest. cpp	两个独立 World 实例运行相同指令序列，验证状态哈希一致
LockstepCommandSync	src/tests/integration/ LockstepCommandSyncTest.cpp	CommandSyncController 的锁步延迟和命令交换
SimulationDriverModes	src/tests/integration/ SimulationDriverModesTest. cpp	三种运行模式（单机/回放/锁步）的正确性
ReplayDeterminism	src/tests/ ReplayDeterminismTest.cpp	录制 → 回放 → 比较状态哈希
MapLoading	src/tests/MapLoadingTest.cpp	地图 JSON 解析、Tilemap 尺寸和地块类型正确性
FogOfWar	src/tests/FogOfWarTest.cpp	迷雾揭示、衰减、多队伍隔离
RegressionBuildPlacement	src/tests/regression/ BuildPlacementRegressionTest. cpp	回归测试：防止建筑放置逻辑退化
LogicNoFloatCheck	tools/check_logic_no_float. py	静态检查：确保 src/logic/ 目录下无 float/double 关键字

9.2 测试结果

运行命令：

```
ctest --test-dir build --output-on-failure
```

结果：

```
100% tests passed, 0 tests failed out of 21
```

```
Total Test time (real) = 0.25 sec
```

全部 21 个测试用例通过，总耗时 0.25 秒。测试结果验证了：

- **确定性保证：** DeterminismSmoke、DualInstanceDeterminism、ReplayDeterminism 三个测试从不同角度验证了模拟的确定性——相同输入始终产生相同状态哈希。
- **数值正确性：** FixedPointMath 测试通过了所有算术运算和边界条件。
- **系统行为正确性：** 建造、生产、战斗、寻路等核心系统的行为符合预期。
- **回归保护：** RegressionBuildPlacement 和 LogicNoFloatCheck 防止关键约束被意外破坏。

10 收获

通过完成本项目，我在以下方面获得了显著提升：

10.1 C++ 工程能力

- 掌握了 CMake 构建系统的使用，包括静态库、可执行文件、条件编译选项的管理。
- 实践了 C++20 新特性：constexpr、[[nodiscard]]、noexcept、designated initializers、std::optional。
- 学会了如何组织一个约 40 个源文件、15+ 个类的中等规模 C++ 项目。

10.2 软件架构设计

- 深入理解了 ECS (Entity-Component-System) 设计模式，体会了“数据与逻辑分离”带来的灵活性和可测试性。
- 实践了分层架构，严格分离逻辑层、渲染层、应用层，确保核心代码不依赖图形库。
- 学会了“组合优于继承”的设计原则，项目中几乎所有类都是 `final` 独立类。

10.3 游戏开发基础

- 实现了固定时间步长的游戏循环 (`GameLoop + TickScheduler`)，理解了帧率无关的模拟更新。
- 掌握了 A* 寻路算法的完整实现，包括启发式函数选择、优先队列优化、边界和障碍物处理。
- 实现了战争迷雾系统，理解了“揭示-衰减”的两阶段模型。
- 实践了等角投影 (`isometric projection`) 的地图渲染。

10.4 算法与数据结构

- 手写实现了 A* 最短路径搜索，使用了最小堆和哈希表进行优化。
- 实现了有限状态机 (FSM) 驱动 AI 行为。
- 使用定点数替代浮点数，确保跨平台确定性。

10.5 测试与质量保证

- 编写了 21 个测试用例，覆盖单元测试、集成测试和回归测试。
- 使用 Python 脚本进行静态代码检查（逻辑层无浮点数约束）。
- 建立了“修改代码 → 编译 → 运行全量测试”的开发工作流。

10.6 调试与问题排查

- 每次构建失败后，通过编译器错误信息定位问题并修复。
- 学会了使用 `git log`、`git diff` 等工具追踪代码变更历史。

11 AI 使用报告

在 *The Chlorophyll Protocol* (叶绿素协议) 项目的研发过程中，依托了多模态 AI 矩阵，实现了高效的系统重构与全栈工程落地：

- **Gemini** 担当顶层架构师，主导了基于 **EnTT** 的 ECS (实体组件系统) 解耦设计，并规范了“阳光与电力”双重资源状态机的逻辑边界与 MVP (最小可行性产品) 玩法闭环的演进路线。
- **DeepSeek-V4Pro** 聚焦于底层确定性算法。针对帧同步网络架构，协助推导并实现了完全规避浮点数精度的 **Fixed32** 定点数数学库，并在 A* 寻路网格映射与基于 `entity_id` 的唯一性索敌仲裁中确保了计算的绝对一致。

- **Kimi K2.6** 专注于渲染表现与交互鲁棒性。高效协助重构了 SFML 多视口裁剪逻辑，根治了游戏实体穿透至建造侧边栏底层的渲染缺陷，并实现了“右键自点击切换重炮模式”的事件拦截。
- **Codex** 作为代码补全利器深度嵌入 workflow，快速生成了刷怪系统与僵尸 AI 有限状态机 (FSM) 的大量模板化代码，大幅缩短了底层组件与系统的落地周期。

本项目的工程实践充分论证了多 AI 协同在硬核技术攻关与复杂业务解耦中的高实用价值。